

+ 26

AEMBER

Modern Embedded C++ Without the Chaos

ARIAN AJDARI



20
26



AEMBER

Modern Embedded C++ Without the Chaos

1.1 MB

PID1 binary

C++26

P2996 reflection

< 10s

boot time

x64 + aarch64

architectures

Arian Ajdari · CppCon 2026 · September 14th, 16:45

About the Code Examples

All code examples are taken directly from the Aember source code.

- ◆ `github.com/ajdariarian94/aember`
- ◆ Some examples are slightly simplified or trimmed for slide readability.
- ◆ The concepts they demonstrate are real.
- ◆ Nothing is invented for the sake of the talk. 😊

INTRODUCTION

The Problem

You want to write embedded application logic.

Instead you spend hours on:

- bootstrapping init systems
- wiring services and dependencies
- debugging startup failures
- fighting tooling not built for embedded

None of this is your application.

What is AEMBER?

A developer-first PID1 – the first process your OS runs.

Process supervision

start, stop, restart with configurable policies

Container orchestration

LXC containers as first-class supervised services

Signal handling

signalfd + epoll – zero coalescing, zero lost SIGCHLD

Health monitoring

heartbeat, RSS, uptime, network – live Qt dashboard

Developer experience

devcontainer + QEMU + one command – 5 min to contributing

THE PID1 CHALLENGE

PID1: What It Is & Why It Matters

PID 1 is the first process. Every other process is its descendant.

- ◆ Receives all orphaned child processes
- ◆ Must reap zombies via `waitpid()`
- ◆ Handles `SIGTERM` on system shutdown
- ◆ If it crashes – kernel panic

aember (PID 1)

aember-child-demo

[reaper jthread]

aember-monitor [LXC]

[signal jthread]

aember-echo [LXC]

[signal jthread]

Architecture: Component Overview

<code>aember-init</code>	PID1 entry point – orchestrates the full init sequence
<code>child-supervisor</code>	Tracks all child PIDs – maps <code>pid_t</code> to service name
<code>container-manager</code>	LXC lifecycle – squashfs mount, overlayfs, start, stop
<code>device-health</code>	Reads <code>/proc</code> stats – RSS, threads, uptime, heartbeat JSON
<code>module-loader</code>	Loads <code>.ko</code> modules at runtime – <code>finit_module()</code> with ordering
<code>mount-manager</code>	Mounts <code>proc</code> , <code>sys</code> , <code>cgroup2</code> , <code>tmpfs</code> – early filesystem setup
<code>network-manager</code>	ICMP ping connectivity check – RTT measurement, status callback
<code>process-manager</code>	<code>fork()+execvp()</code> – restart policy, <code>sigprocmask</code> , <code>ScheduleRestart</code>
<code>root-manager</code>	<code>pivot_root()</code> – switches from <code>initramfs</code> to real <code>rootfs</code>
<code>service-manager</code>	Unified API over processes and containers – state machine

Boot Sequence: initramfs → pivot → root

Two-phase init – pivot to real rootfs before services start.

```
// Phase 1: StartInitramfs()
void AemberInit::StartInitramfs() {
    mount_manager->MountInitramfsFilesystems();
    root_config.ParseFromProcCmdline(); // root=/dev/sda1
    root_manager->PerformPivot(root_config);
    // exec() → /usr/bin/aember --root
}

// Phase 2: StartRoot()
void AemberInit::StartRoot() {
    InitMounts().or_else([&](auto& e) { log_.warn(e.message); });
    InitNetwork().or_else([&](auto& e) { log_.warn(e.message); });
    InitServiceStack();
    LoadAndRegisterProcesses("/etc/aember/services.json");
    LoadAndRegisterContainers("/etc/aember/containers.json");
    service_manager->StartAll();
    monitor_ = std::make_unique<AemberMonitor>();
    heartbeat_.emplace([this](const auto& p) { OnHeartbeat(p); });
    RunLoop();
}
```

Container Lifecycle: squashfs → OverlayFS → LXC

Read-only squashfs + writable overlay = immutable containers with state.

01 **Mount squashfs**
read-only lower layer via loop device

02 **OverlayFS**
writable upper layer over squashfs

03 **Configure LXC**
namespaces, cgroups, device access

04 **Start + Track**
lxc_start() → AddChild(init_pid)

05 **On exit**
reap → MirrorState → ScheduleRestart

06 **Restart**
new squashfs mount → new PID tracked

Service State Machine

Every process and container transitions through the same states.

stopped

starting

running

restarting

failed

stopping

```
// MirrorState keeps ServiceManager::states_ in sync
void ServiceManager::MirrorState(
    std::string_view name,
    ProcessState old_state,
    ProcessState new_state) {
    states_[std::string{name}] = new_state;
    if (state_change_cb_)
        state_change_cb_(name, old_state, new_state);
}
```

C++23/26 FEATURES IN ACTION

Logging: Type-Safe spdlog Wrapper

Every class owns its logger. Consistent naming. Zero overhead.

```
class Logger {
public:
    explicit Logger(std::string_view name)
        : logger_(spdlog::get(std::string{name})) {
        if (!logger_)
            logger_ = spdlog::stdout_color_mt(std::string{name}); }

    template <typename... Args>
    void info(fmt::format_string<Args...> fmt, Args&&... args) const {
        logger_>info(fmt, std::forward<Args>(args)...);
    }
    template <typename... Args>
    void error(fmt::format_string<Args...> fmt, Args&&... args) const {
        logger_>error(fmt, std::forward<Args>(args)...);
    }
private:
    std::shared_ptr<spdlog::logger> logger_;
};
```

Deducing this – Zero-Overhead Policy Classes

C++23: explicit object parameter. CRTP without templates. Perfect for chaining.

```
//  Deducing this – C++23, no quadruplication of code needed
class Manager {
    // Const-correct getter – one definition covers const and non-const
    template <typename Self>
    auto& GetLogger(this Self&& self) { return self.log_; }

    // std::forward_like propagates value category of Self
    template <typename Self>
    decltype(auto) GetConfig(this Self&& self) {
        return std::forward_like<Self>(self.config_); }
};
```

std::expected – Exception-Free Error Handling

No try/catch anywhere in PID1. Errors are values. Chains compose.

```
using Result = std::expected<void, Error>;

Result AemberInit::InitMounts() {
    if (!mount_manager_>MountEarlyFilesystems())
        return std::unexpected(Error{"Some mounts failed"});
    return {};}

// Full monadic chain – errors propagate automatically
// Each step only runs if the previous succeeded
InitMounts()
    .and_then([&]() { return InitNetwork(); }) // ← only if mounts ok
    .and_then([&]() { return InitServiceStack(); }) // ← only if network ok
    .and_then([&]() { return LoadServices(); }) // ← only if stack ok
    .or_else([&](const Error& e) -> Result {
        log_.error("Init failed: {}", e.message);
        return {};} // continue gracefully
    );

// transform – convert the value if present
auto pid = process_manager_>Start(name).value_or(-1); // -1 if Start() failed
```

Lambda Captures — Precise Ownership

Capture exactly what you need. No accidental dangling references.

```
// ScheduleRestart — this + key + delay by value
// "this" needed for on_restart_member access
std::jthread{[weak = weak_from_this(), key, delay](std::stop_token st) {
    std::this_thread::sleep_for(delay);
    if (st.stop_requested()) return;

    if (auto self = weak.lock())
        if (self->on_restart_) self->on_restart_(key); // member via this
}}.detach();

// NetworkManager — this only
network_manager_ = std::make_unique<NetworkManager>(cfg,
    [this](const ConnectivityStatus& s) {
        OnNetworkStatus(s);
    });

// ContainerManager — this + state parameters by value
container_manager_ = std::make_shared<ContainerManager>(mount_manager_,
    [this, name](
        ContainerState old_state,
        ContainerState new_state) {
        OnContainerStateChange(name, old_state, new_state); });
```

Heartbeat: Live Dashboard Feed

Every second: PID1 stats + network + service states → JSON file.

```
void AemberInit::OnHeartbeat(const nlohmann::json& base) {
    auto payload = monitor_>Snapshot(); // PID1 /proc stats
    auto status = network_manager_>GetStatus();

    payload["network"] = {
        {"online", status.online},
        {"rtt_ms", status.rtt_ms},
    };

    auto services = nlohmann::json::array();
    for (const auto& name : service_manager_>GetNames())
        services.push_back({
            {"name", name},
            {"state", ToString(service_manager_>GetState(name))},
            {"type", service_manager_>IsContainer(name) ? "container" : "process"},
        });
    payload["services"] = services;
    payload.merge_patch(base);
    monitor_>LogToFile(payload); // → /var/log/aember-monitor.json
}
```

std::move_only_function – Owing Callbacks

C++23: non-copyable callable. Captures ownership. No shared_ptr needed.

```
class ProcessManager {
public:
    using RestartCallback =
        std::move_only_function<void(const std::string&);>;

    void SetRestartCallback(RestartCallback cb) {
        on_restart_ = std::move(cb); }

private:
    RestartCallback on_restart_;
};

// Wired in InitServiceStack() – routes restarts through ServiceManager
process_manager_->SetRestartCallback(
    [this](const std::string& name) {
        service_manager_->Start(name); // new PID → ChildSupervisor
    }
);

// Can't copy – ownership is explicit
// RestartCallback a = on_restart_; // ← compile error
```

Designated Initializers – Self-Documenting Config

C++20: named fields in aggregate init. No positional confusion.

```
struct ProcessConfig {
    std::string      name;
    std::string      executable;
    std::vector<std::string> args;
    bool restart_on_failure{true};
    int  max_restarts{5};
    std::chrono::milliseconds restart_delay{5000ms};
};

// ❌ Positional – which bool is which?
ProcessConfig cfg{"demo", "/usr/bin/demo", {}, true, 3, 1000ms};

// ✅ Designated – self-documenting
ProcessConfig cfg{
    .name           = "aember-child-demo",
    .executable     = "/usr/bin/aember-child-demo",
    .restart_on_failure = true,
    .max_restarts   = 3,
    .restart_delay   = 1000ms,
};
```

Concepts – Constrained Callbacks

C++20: enforce callback signatures at compile time. Better errors.

```
// Concept: any callable that accepts a JSON payload
template <typename F>
concept HeartbeatCallable =
    std::invocable<F, const nlohmann::json>;

// Used in Heartbeat – constrained template constructor
class Heartbeat {
public:
    template <HeartbeatCallable F>
    explicit Heartbeat(F&& cb)
        : callback_(std::forward<F>(cb)) {}
private:
    std::move_only_function<void(const nlohmann::json)> callback_;
};

// ✅ Lambda – satisfies HeartbeatCallable
Heartbeat hb{[] (const nlohmann::json& p) {
    OnHeartbeat(p);
}};

// ❌ Wrong signature – compile error
Heartbeat hb{[] (int x) {}}; // error: int is not json&
```

Structured Bindings – Readable Iteration

C++17: name both parts of a pair. No `.first/.second` noise.

```
// ChildSupervisor – map<pid_t, string>
// ❌ Without structured bindings
for (const auto& pair : children_)
    log_.warn("  '{}' (pid={})", pair.second, pair.first);

// ✅ Structured bindings – intent is clear
for (const auto& [pid, name] : children_)
    log_.warn("  '{}' (pid={})", name, pid);

// In ProcessManager environment setup
for (const auto& [key, value] : config.environment)
    setenv(key.c_str(), value.c_str(), 1);

// In if – binding + condition in one line
if (const auto [it, inserted] = containers_.emplace(name, entry); !inserted)
    log_.warn("Container '{}' already registered", name);
```

C++26 P2996 – Reflecting Over Any Struct

Iterate fields at compile time. Print name, type, and value – zero runtime overhead.

```
struct ServiceContainer { const char* path; std::size_t restart; };
constexpr ServiceContainer serviceContainer{ .path = "/usr/bin/aember", .restart = 0 };

template <std::meta::info Type>
void print_fields(auto Instance) {
    // Reflect all non-static data members at compile time
    constexpr static auto reflectedFields =
        std::define_static_array(
            std::meta::nonstatic_data_members_of(Type,
            std::meta::access_context::current())));

    // Compile-time for – expands at compile time, not runtime
    template for (constexpr auto member : reflectedFields) {
        std::println("{}: {} = {}",
            std::meta::identifier_of(member),          // field name
            std::meta::display_string_of(std::meta::type_of(member)), // type as string
            Instance.[member]);                        // field value via splice
    }
}

print_fields<^^ServiceContainer>(serviceContainer);
```

if consteval – Compile-Time Optimization Paths

C++23: different code path at compile time vs runtime. Zero overhead.

```
template <typename T>
constexpr std::string_view component_name() {
    if constexpr {
        // Compile time: reflect the type name directly
        return std::meta::identifier_of^^T;
    } else {
        // Runtime: return pre-computed string
        return typeid(T).name();
    }
}

// C++26 P2996: type name at compile time
constexpr auto kMonitorName = component_name<AemberMonitorClient>();

// Used in aember-child-demo:
log_.info("Monitor type: {} (C++26 P2996)", kMonitorName);
```

ARCHITECTURE DEEP DIVE

Process Restart: The Full Chain

kill → SIGCHLD → HandleExit → ScheduleRestart → RestartCallback → Start()

```
bool ProcessManager::HandleExit(std::string_view s_name, pid_t pid, int status) {
    const std::string name = std::string(s_name);
    const bool was_stopping = (states_[name] == ProcessState::Stopping);
    states_[name] = was_stopping
        ? ProcessState::Stopped : ProcessState::Failed;

    if (on_exit_) on_exit_(name, pid, status);
    if (!was_stopping) ScheduleRestart(name);
    return true; }

void ProcessManager::ScheduleRestart(std::string_view s_name) {
    const std::string name = std::string(s_name);
    if (++restart_counts_[name] > config_.max_restarts) {
        states_[name] = ProcessState::Failed;
        return; } // ← braces required — return is inside if
    states_[name] = ProcessState::Restarting;

    std::jthread{[weak = weak_from_this(), key = name, delay = config_.restart_delay](std::stop_token st) {
        std::this_thread::sleep_for(delay);
        if (st.stop_requested()) return;
        if (auto self = weak.lock())
            if (self->on_restart_) self->on_restart_(key);
    }}.detach(); }
```

Process Spawning: `fork()` + `execvp()`

Wrapped in `SpawnProcess()`. Critical: unblock signals before `exec`.

```
[[nodiscard]] std::optional<pid_t> ProcessManager::SpawnProcess(
    const ProcessConfig& config) {
    const pid_t pid = fork();
    if (pid < 0) return std::nullopt;
    if (pid == 0) { // child
        sigset_t all; sigfillset(&all);
        sigprocmask(SIG_UNBLOCK, &all, nullptr);

        for (const auto& [k, v] : config.environment) setenv(k.c_str(), v.c_str(), 1);

        // Build null-terminated argv
        std::vector<char*> argv;
        argv.push_back(const_cast<char*>(config.executable.c_str()));
        for (const auto& a : config.args)
            argv.push_back(const_cast<char*>(a.c_str()));
        argv.push_back(nullptr);
        execvp(argv[0], argv.data()); _exit(1);
    }
    return pid;}
```

Zombie Reaping: Dual Strategy

signalfd for speed. Reaper jthread for reliability.

```
// Strategy 1: SIGCHLD handler - instant
signalfd_loop_handler_.Register(SIGCHLD, [this](int) {
    int status = 0; pid_t pid;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0)
        service_manager_->HandleExit(pid, status);
});

// Strategy 2: Reaper jthread - catches what signalfd missed
// (LXC fires its own SIGCHLD - can race with our handler)
reaper_thread_ = std::jthread{[this](std::stop_token st) {
    while (!st.stop_requested()) {
        std::this_thread::sleep_for(1s);
        pid_t pid; int status;
        while ((pid = waitpid(-1, &status, WNOHANG)) > 0)
            service_manager_->HandleExit(pid, status);
    }
}
};

// Result: zero zombies. Tested under rapid kill -9 cycles.
```

signalfd + epoll – No Lost SIGCHLD

Wrapped in a C++ class. jthread owns the loop. stop_token cancels instantly.

```
class SignalHandler {
    using Callback = std::move_only_function<void(int)>;
    void Register(int signal, Callback cb) {
        callbacks_[signal] = std::move(cb); sigaddset(&mask_, signal); }

    void Start() {
        signal_fd_ = signalfd(-1, &mask_, SFD_NONBLOCK | SFD_CLOEXEC);
        epoll_fd_ = epoll_create1(EPOLL_CLOEXEC);
        signal_thread_ = std::jthread{[this](std::stop_token st) {
            while (!st.stop_requested()) {
                epoll_wait(epoll_fd_, events, kMax, -1);
                signalfd_siginfo info{};
                while (read(signal_fd_, &info, sizeof(info)) == sizeof(info))
                    callbacks_[info.ssi_signo](info.ssi_signo);
            } } };

        int signal_fd_{-1}, epoll_fd_{-1};
        sigset_t mask_{};
        std::jthread signal_thread_;
        std::map<int, Callback> callbacks_;
    };
};
```

Pipe Operator + ranges::to – C++23 Pipelines

Compose views lazily. Collect with ranges::to

```
// Get names of all running services
auto running_names =
    service_manager_>GetNames()
    | std::views::filter([&](const auto& name) {
        return service_manager_>GetState(name)
            == ProcessState::Running;})
    | std::ranges::to<std::vector<std::string>>();

// Build JSON services array
// ranges::to<nlohmann::json> invalid – json is not a range container
// ✅ Use vector<json>, then assign
auto services =
    service_manager_>GetNames()
    | std::views::transform([this](const auto& name) {
        return nlohmann::json{
            {"name", name},
            {"state", ToString(service_manager_>GetState(name))},
        };
    })
    | std::ranges::to<std::vector<nlohmann::json>>();

payload["services"] = services; // assign vector → json array
```

Composable Service Registration

Filter, transform, collect – no raw loops, no intermediate vectors.

```
// Register all processes from JSON config
const auto registered =
    configs
    | std::views::filter([&](const auto& cfg) {
        if (!process_manager_>AddProcess(cfg)) {
            log_.error("Failed to register '{}'", cfg.name); return false;
        }
        return true;
    })
    | std::views::transform([](const auto& cfg) {
        return cfg.name; // extract name of registered ones
    })
    | std::ranges::to<std::vector<std::string>>();

// Then add to ServiceManager
std::ranges::for_each(registered, [&](const auto& name) {
    service_manager_>AddProcess(name);
});
```

std::string_view – Zero-Copy Strings

No allocation. No copy. "PID1 never allocates for string passing"

```
// ❌ forces heap allocation when passing string literal ("...")
void AddProcess(const std::string& name);

// ✅ std::string_view – non-owning, zero copy
void AddProcess(std::string_view name);

// Used everywhere in AEMBER:
class ProcessManager {
    [[nodiscard]] std::optional<pid_t> Start(std::string_view name);
    bool Stop(std::string_view name);
    bool AddProcess(const ProcessConfig& config);
    void ScheduleRestart(std::string_view name);
};

// Caller – no allocation, no lifetime issues
process_manager_>Start("aember-child-demo"); // string literal → view
process_manager_>Start(name);                 // std::string → view
```

[[nodiscard]] – Enforced Return Value Checking

Compiler error if you ignore a PID or error. No silent failures.

```
class ProcessManager {
    // Ignoring the PID is almost always a bug
    [[nodiscard]] std::optional<pid_t> Start(std::string_view name);

    // Ignoring mount failure silently is dangerous
    [[nodiscard]] bool MountEarlyFilesystems();
};

// ❌ Silently ignoring – compiler warning/error
process_manager_->Start("demo"); // warning: ignoring [[nodiscard]]

// ✅ Explicit handling
if (auto pid = process_manager_->Start("demo")) {
    child_supervisor_.AddChild(*pid, "demo");
} else {
    log_.error("Failed to start demo");
}

// Also used on std::expected returns – must handle errors
```

std::optional – Nullable Without Null Pointers

Module path lookup, PID lookup – no raw pointers, no sentinel -1.

```
// Find .ko or .ko.zst path – may not exist
std::optional<std::filesystem::path> ModuleLoader::FindModule(
    std::string_view name) const {
    for (const auto& entry : std::filesystem::recursive_directory_iterator(modules_dir_)) {
        auto stem = entry.path().stem().string();

        if (stem.ends_with(".ko")) stem.resize(stem.length() - 3);
        if (stem == name) return entry.path();
    }
    return std::nullopt;
}

// Caller – forced to handle missing case
auto path = FindModule(name);
if (!path) { log_.warn("Module '{}' not found", name); }
```

std::chrono – Type-Safe Durations

Duration literals. No "is this ms or seconds?" bugs.

```
using namespace std::chrono_literals;

struct ProcessConfig {
    // ✅ Type encodes the unit – impossible to mix up
    std::chrono::milliseconds restart_delay{5000ms};
};

// Heartbeat interval – 1 second, readable
Heartbeat hb{callback, 1000ms};

// AemberMonitor uptime – steady_clock, no drift
const auto session_uptime =
    std::chrono::duration_cast<std::chrono::seconds>(
        std::chrono::steady_clock::now() - start_time_).count();
```

std::atomic – Lock-Free State Flags

running_, connected_ – no mutex for simple boolean checks.

```
class SignalHandler {
    std::atomic<bool> running_{false};
    int wake_pipe_{-1,-1};

    void Start() {
        // exchange returns old value – prevents double-start
        if (running_.exchange(true)) return;
        ::pipe2(wake_pipe_, O_CLOEXEC | O_NONBLOCK);

        // register signal_fd_ and wakeup read end
        signal_thread_ = std::jthread{...};
    }

    void Stop() {
        if (!running_.exchange(false)) return;
        signal_thread_.request_stop();
        char byte = 'q';
        ::write(wake_pipe_[1], &byte, 1); // unblock epoll_wait
    }
};
```

std::variant – Type-Safe Union for Service Config

Process or container – stored in one list, dispatched safely.

```
// Service is either a process or a container
template<class... Ts> struct overloaded : Ts... {using Ts::operator()...};

using ServiceConfig =
    std::variant<ProcessConfig, ContainerConfig>;

// std::visit – exhaustive dispatch, compile-time checked
void ServiceManager::Start(std::string_view name) {
    std::visit(overloaded{
        [&](const ProcessConfig& cfg) {
            process_manager_.Start(name, cfg);},
        [&](const ContainerConfig& cfg) {
            container_manager_.Start(name, cfg);}
    }, services_.at(name));
}

std::unordered_map<std::string, ServiceConfig> services_;

// No if/else chains. No dynamic_cast. Compiler enforces exhaustiveness.
```

PERFORMANCE & CONSTRAINTS

Performance & Constraints

1.1 MB

PID1 binary
aarch64 Release

6.6 MB

RSS at runtime
all services running

< 10s

Full boot time
kernel to services ready

0

Exceptions thrown
std::expected throughout

Compared to:

systemd

~30-50 MB RSS · 1.5M lines of C · monolithic — init+logging+DNS+network+...

Pantavisor

"1MB core" (marketing) · C

AEMBER

1.1MB binary · 6.6MB RSS · C++26 · modular · real measured numbers

LIVE DEMO

LIVE DEMO

- ◆ AEMBER PID1 booting in QEMU (< 10 seconds)
- ◆ Qt dashboard [LXC container] showing live data
- ◆ Kill aember-child-demo → state: running → failed → restarting → running
- ◆ All states visible in the Qt dashboard in real time

TAKEAWAYS & Q&A

Key Takeaways

01

C++23/26 is production-ready for embedded

`std::expected`, `jthread`, `stop_token`, `move_only_function`, P2996

02

signalfd beats sigwait for PID1

file descriptor events – no coalescing, drain all pending signals

03

Dual reaping strategy works

signalfd for speed + reaper `jthread` for reliability

04

LXC + squashfs is lightweight embedded containers

immutable base + writable overlay – minimal overhead

05

Developer experience is a feature

devcontainer + QEMU = contributors productive in 5 minutes

06

Open source – services not yet written

OTA, MQTT, fleet management – your contribution opportunity

Developer Experience

From git clone to running QEMU in under 5 minutes.

```
# 1. Clone
$ git clone https://github.com/ajdariarian94/aember

# 2. One script — Docker, X11, submodules
$ ./tools/scripts/prerequisites.sh

# 3. VS Code → "Reopen in Container" → ready
$ code .

# 4. Build
$ aember app build

# 5. Pack artifacts
$ aember app pack

# 6. Launch in QEMU with virtio-gpu display
$ aember app launch
```

devcontainer

all tools preinstalled

clangd

IntelliSense + format on save

QEMU

virtio-gpu · instant simulation

GDB

gdb-multiarch for aarch64

GitHub Releases

prebuilt artifacts — no Yocto

Tests in VS Code

testmate

Thank You

Questions?

GitHub: github.com/ajdariarian94/aember

Session: cppcon2026.sched.com/event/2RT4v

September 14th: 16:45 · Aurora, CO



*"The most critical process on your system deserves
the most expressive, safest language available."*